

Problem Set 0: Introduction to Python

The goal of this problem set is to test your ability to code simple python programs and understand python code. In addition, you should use this lab as a chance to get familiar with autograder which will automatically grade your code.

To run the autograder, type the following command in the terminal:

```
python autograder.py
```

If you wish to run a certain problem only (e.g. problem 1), type:

```
python autograder.py -q 1
```

where 1 is the number of the problem you wish to run.

You can also specify a single testcase only (e.g. testcases01.json in problem 1) by typing:

```
python autograder.py -q 1/test1.json
```

To debug your code through the autograder, you should disable the timeout functionality. This can be done via the `debug` flag as follow:

```
python autograder.py -d -q 1/test1.json
```

Or you could set a time scale to increase or decrease your time limit. For example, to half your time limits, type:

```
python autograder.py -t 0.5 -q 1/test1.json
```

Note: Your machine may be faster or slower than the grading device. To automatically detect your machine's speed, the autograder will run `speed_test.py` to measure your machine relative speed, then it will scale the time limits automatically. The speed test result is automatically stored in `time_config.json` to avoid running the speed test every time you run the autograder. If you want to re-calculate your machine's speed, you can do so by either running `speed_test.py`, or deleting `time_config.json` followed by running the autograder.

Instructions

In the attached python files, you will find locations marked with:

```
#TODO: ADD YOUR CODE HERE
utils.NotImplemented()
```

Remove the `utils.NotImplemented()` call and write your solution to the problem. **DO NOT MODIFY ANY OTHER CODE**; The grading of the assignment will be automated and any code written outside the assigned locations will not be included during the grading process.

IMPORTANT: You can only use the **Built-in Python Modules**. Do not use external libraries such as `Numpy`, `Scipy`, etc. You can check if a module is builtin or not by looking up [The Python Standard Library](#) page.

Introduction to Type hints

In the given code, we heavily use [Type Hints](#). Although Python is a dynamically typed language, there are many reasons that we would want to define the type for each variable:

1. Type hints tell other programmers what you intend to put in each variable.
2. It helps the intellisense find reasonable completions and suggestions for you.

Type hints are defined as follows:

```
variable_name: type_hint
```

for example:

```
# x will contain an int
x: int

# x contains a string
x: str = "hello"

# l contains a list of anything
l: List[Any] = [1, 2, "hello"]

# u contains a float or a None
u: Union[float, None] = 1.25
```

We can also write type hints for functions as follows:

```
# Function is_odd takes an int and returns a bool
def is_odd(x: int) -> bool:
    return (x%2) == 1
```

Some type hints must be imported from the package `typing`. Such as:

```
from typing import List, Any, Union, Tuple, Dict
```

In all of the assignments, we will use type hints whenever possible and we recommend that you use them for function and class definitions.

Problem 1: Anagrams

Inside `anagram_check.py`, modify the function `anagram_check` to return **true** if and only if the two input strings are anagrams of each other.

An anagram is formed when two strings contain exactly the same letters in any order.

- For example, "listen" and "silent" are anagrams since they both contain the letters l, i, s, t, e, n.
- "hello" and "world" are not anagrams since their letters differ.
- "apple" and "papel" are anagrams, while "apple" and "pale" are not, because "pale" is missing one 'p'.

Assume the comparison is **case-sensitive** (so "Listen" and "Silent" are **not** considered anagrams).

Problem 2: Word Histogram

Inside `word_histogram.py`, modify the function `word_histogram` to return a dictionary containing each word in the input string alongside its frequency.

A word histogram counts how many times each word appears in a given text.

- For example, "cat dog cat" should return {"cat": 2, "dog": 1}.
- "apple banana apple orange banana" should return {"apple": 2, "banana": 2, "orange": 1}.
- An empty string should return an empty dictionary {}.

Assume that words are separated by spaces, and the comparison is **case-sensitive** (so "Dog" and "dog" are treated as different words).

Problem 3: Exam Score Calculator with Score and Grade

Inside `exam_score.py`, modify the function `calculate_exam_score` that, given a list of exam questions and a student's answers, returns **both the raw numeric score and the letter grade**.

Each question has a certain number of points (weight), and the student only earns those points if their answer is correct.

After calculating the total score, convert it into a letter grade using a helper function `score_to_grade`. The grading scale is as follows:

- 90 and above → "A"
- 80–89 → "B"
- 70–79 → "C"

- 60–69 → "D"
- Below 60 → "F"

For example:

- If the exam contains 3 questions with weights `[50, 30, 20]` and the student answers correctly `[True, True, False]`, the raw score is 80.
- The function should return `(80, "B")`.

Problem 4: Most Frequent Item in 2D Grid

Inside `most_frequent.py`, modify the function `most_frequent_item` to find the value that appears most frequently in a `Grid` object.

The function should return the value that occurs the most in the grid.

If multiple values have the same highest frequency, you may return any one of them.

Example

If the grid contains:

```
[ [1, 2, 3], [4, 2, 6], [7, 8, 2] ]
```

The function should return 2 since it appears three times, more than any other number.

Problem 5: Caesar Decipher

A Caesar cipher works by shifting each letter in the text along the alphabet by a specific amount which is the shift. For example, if the shift is 2 to the right, then 'a' will become 'c', 'b' will become 'd', 'c' will become 'e', and so on. So if you cipher the word 'fez' by a shift of 2 to the right, it will become 'hgb'. Notice that 'z' wrapped around and became 'b'. Since we know that the cipher was created by shifting 2 to the right, we can decipher the result by shifting 2 to the left, and 'hgb' will become 'fez'.

However, if we don't know the shift used for the cipher, we can't decipher the output, right? Well, there is something we can do. If we expect the original text to be english, we can try every possible shift (there are only 26 possible shift, if we count shifting with 0 as an option), and see which one create the more english words. So the algorithm can be as follows: Using each possible shift, decipher the ciphered text, and count the number of words that are not in the dictionary. Return the deciphered text with the least number of words out of the dictionary.

Inside `caesar.py`, modify the function `caesar_decipher`. This function takes the ciphered text (string) and the dictionary (a list of strings where each string is a word). It should return a tuple containing:

- The deciphered text (string).
- The shift of the cipher (non-negative integer). Assume that the shift is always to the right (in the direction from 'a' to 'b' to 'c' and so on). So if you return 1, that means that the text was ciphered by shifting it 1 to the right, and that you deciphered the text by shifting it 1 to the left.
- The number of words in the deciphered text that are not in the dictionary (non-negative integer).

Important Note: The ciphered text will only contain spaces and lower-case english letters. There will be no numbers, punctuations, etc. In addition, there will be one and only one space character between each pair of

neighboring words. For the dictionary, all the words in it will only contain lower-case english letters.

Delivery

IMPORTANT: You must fill the `student_info.json` file since it will be used to identify you as the owner of this work. The most important field is the `id` which will be used by the automatic grader to identify you. You also must compress the solved python files and the `student_info.json` file together in a **zip** archive so that the autograder can associate your solution files with the correct `student_info.json` file. The failure to abide with the these requirements will lead to a zero since your submission will not be graded.

You should submit a **zip** archive containing the following files:

1. `student_info.json`
2. `anagram_check.py`
3. `word_histogram.py`
4. `exam_score.py`
5. `most_frequent.py`
6. `caesar.py`

The delivery deadline is `October 10th 2026 23:59`. It should be delivered on **Google Classroom**. This is an individual assignment. The delivered code should be solely written by the student who delivered it. Any evidence of plagiarism will lead to receiving **zero** points.